Executive Architecture Design Document

AI-Powered Textbook-to-Video Learning Platform

Hackathon: Build in AI for India **Version:** 3.0.0 **Status:** Approved for Development

Prepared By: Team 4NLPians **Team Members:** Bharath M · Harish Sankaranarayanan · Saravanan Nallamuthu · Shanmugasundaram

---

Table of Contents

---

1. Executive Summary

This platform converts static government-school textbook chapters into short, localized, animated educational videos, using AI once at content-creation time and never again at playback time.

The problem

Students in Indian government schools move between a home language and English scientific vocabulary with little support in between. Printed textbooks don't bridge that gap, and hand-made animated lessons are too slow and too expensive to produce at scale.

The approach

A linear, deterministic pipeline reads a textbook chapter, writes a localized script, generates narration and subtitles, and renders a video. The video is then served to every student in that class for free, with no further AI cost.

Why this works architecturally

The system separates content creation (AI-heavy, runs once per chapter) from content consumption (AI-free, runs for every student). This is the single decision that makes the product affordable at public-school scale, and every other architectural choice in this document supports it.

How this document is organized

The sections that follow move from product vision, to platform-wide architecture, to each subsystem in detail (mobile client, backend, AI orchestration, prompt and skills layer, tooling), and close with engineering standards, the delivery plan, and the key decisions behind the design.

2. Product Vision

Vision. Give every child in India access to high-quality conceptual video lessons, regardless of region or income.

Mission. Turn a standard textbook chapter into an engaging, culturally grounded video lesson automatically.

Design philosophy. Keep the app radically simple: no nested menus, high contrast, nothing on screen that doesn't help a Class 6 student learn.

Design Principles

- **Child-first.** Every screen and timing decision is built around a Class 6 attention span and reading level.
- **Simple over clever.** No autonomous or unpredictable execution logic — a straightforward pipeline beats a smart one nobody can debug at 2 a.m. during a hackathon.
- **Deterministic.** The same chapter always produces a video of the same structure and quality.
- **Modular.** Layers are separated cleanly enough that contributors can build them in parallel.
- **Offline-friendly.** Rendered video is cached and plays smoothly when the school network drops.
- **Observable.** Telemetry is built in from day one, not bolted on later.
- **Open-source first.** Every dependency can be run locally, for free.
- **Generate once, reuse forever.** All AI cost happens once, during creation — never during playback.
- **Affordable AI.** No vector databases, no multi-turn agent loops at read time.
- **Maintainable.** Strict typing, linting, and tests are enforced on every pull request, not treated as optional polish.

Explicitly Out of Scope (MVP)

Non-Goal	Reason
Learning Management System	No grading, profiles, or assignment tracking in v1.
Chat-style AI tutor	No open-ended conversation surface for students.
Adaptive tutoring / human fallback	Out of scope until the core pipeline is proven.
Curriculum management tooling	Textbook structure is fixed input, not an authored asset.
Autonomous web research	The system only reads the provided textbook — nothing else.
General-purpose AI assistant	This is a single-purpose video generator, not a chatbot.

The Core Innovation: Decoupling Creation from Consumption

Diagram 1 — Content creation runs once per chapter; content consumption runs free for every student.

Because rendering happens once per chapter and streaming is free forever after, cost does not grow with the number of students — only with the number of chapters. This is what makes the model viable for underfunded public schools.

Student Journey

Diagram 2 — End-to-end sequence from student request to finished video playback.

3. Platform Architecture

The platform is organized into nine layers, from the mobile client down to the rendered output file. Every diagram in this document uses the same layer names, so a name in one diagram always means the same thing in another.

Diagram 3 — The nine architectural layers, top to bottom.

Layer	Responsibility
Presentation	Renders the UI, collects input, and listens for progress updates.
API Gateway	Validates requests and hands work to background execution — never blocks the request thread.
AI Orchestration	Owns a single shared state object and moves it through the pipeline in a fixed order.
AI Agent	Each agent makes one decision and calls the Skills it needs to act on it.
Skills	Reusable business logic: prompt assembly, response parsing, validation.
LLM Provider	A single interface any Skill can call, regardless of which model answers it.
MCP Execution	Wraps every external tool (TTS, video encoder, parser) behind one async contract.
Infrastructure	The actual disk, subprocess, and network calls.
Output Asset	The final markdown, subtitles, and video file the student receives.

4. System Context and Containers (C4)

4.1 System Context

Diagram 4 — System context: student, client, backend, orchestration engine, and downstream tools/providers.

4.2 Containers and Technology Choices

Container	Technology	Alternatives Considered	Why This Choice	Trade-off & Mitigation
Frontend	Flutter + Riverpod	React Native, native apps	Smooth UI on low-end Android hardware from one codebase.	Larger app size → mitigated with aggressive tree-shaking.
Backend	Python 3.12 + FastAPI	Node.js, Go	Native bindings to the AI/media tooling this pipeline depends on.	Higher memory footprint → mitigated by container memory limits.
AI Orchestration	LangGraph	LCEL, CrewAI	A fixed, linear state machine — no risk of runaway agent loops.	More upfront boilerplate → mitigated with a single graph-construction file.
Rendering	MoviePy + FFmpeg	Blender API, OpenCV	Drop-in multi-track composition without a GPU rendering stack.	CPU-intensive → capped at 720p/24fps with fast-start encoding.

5. AI Video Generation Pipeline

The pipeline runs as one linear pass per chapter — no branching, no retries beyond simple error propagation. This determinism is intentional and should not be relaxed into a general agent loop: a predictable pipeline is easier to debug, cheaper to run, and safer to hand off to new contributors.

Diagram 5 — The five stage groups of the generation pipeline, executed in order.

Stage Group	Responsibility
Intake	Extract text and tables from the PDF into clean Markdown.
Pedagogy	Filter to Class 6 level, apply Indian regional analogies, write the narration script.
Storyboarding	Turn narration into scene-by-scene visual prompts with timing.
Audio & Sync	Synthesize speech, capture word-level timestamps, generate subtitles.
Assembly	Composite video, audio, and subtitle tracks; encode to a streaming-ready MP4.

Multilingual note. Swapping the narration language is a configuration change to the TTS call, not a pipeline change — the design already isolates language as a parameter.

6. UI/UX Philosophy

The app is built for a specific, constrained context: young students, variable networks, low-end devices.

- **No nested navigation.** Everything is a large, color-coded card: pick class, subject, chapter, watch.

- **High-contrast, oversized type.** 24pt bold titles, 16pt body text, matching textbook readability norms.
- **Plain-language progress.** SSE-driven progress bars say "Writing Script..." — never a technical status code.
- **Persistent subtitles.** A high-contrast lower-third bar keeps captions anchored at natural reading eye level.

Shared UI atoms enforce this consistently:

- **AdaptiveSubjectCard** — debounced, tap-safe navigation cards
- **AsyncProgressBar** — typed state → label mapping, no generic spinners
- **AccessibleText** — WCAG AAA contrast and line spacing by default

7. Flutter Client Architecture

The client follows a standard feature-first Clean Architecture split.

Diagram 6 — Presentation, Domain, and Data layers of the Flutter client.

Presentation

Widgets are stateless; they watch `AsyncNotifierProviders` and render `AsyncValue<T>` directly:

```
ref.watch(videoGenerationProvider(taskId)).when(
  data: (video) => VideoPlayerView(asset: video.localPath),
  loading: () => AsyncProgressBar(status: ref.watch(progressProvider)),
  error: (err, stack) => AccessibleErrorWidget(message: err.toString()),
);
```

Domain

Pure Dart, zero third-party dependencies:

```
abstract class IVideoRepository {
  Future<Either<Failure, VideoJobEntity>> requestVideoGeneration({
    required VideoGenerationRequestParams params,
  });
  Stream<VideoStatusUpdateEntity> watchGenerationProgress({required String taskId});
  Future<Either<Failure, List<VideoJobEntity>>> getOfflineCachedVideos();
}
```

Data

A `Dio` client with interceptors maps 401/503/timeouts into domain `Failure` types. `Isar` is the local cache for video paths, manifests, and offline availability.

Routing

GoRouter with named, type-safe routes and deep-link support. No raw string routes.

Responsive layout

Layout dimensions are never hardcoded. A single `ResponsiveLayoutGate` switches between phone and tablet bodies at a 600px breakpoint.

8. Backend Architecture: FastAPI + Async Core

Diagram 7 — FastAPI routers hand off to the LangGraph orchestrator, which reaches infrastructure through three ports.

Async-only rule. No route performs heavy computation inline. Long-running work (parsing, TTS, rendering) is dispatched as a background task the moment the request is accepted; all disk access goes through aiofiles, never the blocking open().

This is a hard rule, not a preference: one blocking call in the request path stalls every other student's request on the same worker.

9. AI Orchestration Layer (LangGraph)

Agents do not loop or make open-ended decisions. Each agent performs one transformation on a shared, typed state object and hands off to the next.

Diagram 8 — The eight-agent linear execution chain.

Agent	Responsibility
Parser	Converts the PDF to Markdown via MarkItDown.
Curriculum	Extracts key concepts and structure from the Markdown.
Lesson Planning	Applies Class 6 pacing and content-density constraints.
Teacher	Writes the localized narration script.
Storyboard	Produces scene-by-scene visual prompts and timing.
Narration	Calls text-to-speech and captures word-level timing.
Video Rendering	Hands the timeline to MoviePy and FFmpeg.
Publishing	Validates the output, updates the cache manifest, and closes the SSE stream.

Shared State (Pydantic)

```
class VideoGenerationState(BaseModel):
    file_path: str
    markdown_content: Optional[str] = None
    sections: List[ChapterSection] = Field(default_factory=list)
    storyboard_beats: List[ScriptBeat] = Field(default_factory=list)
    output_video_path: Optional[str] = None
    error_message: Optional[str] = None
```

Every node reads this state, returns a partial update, and never mutates state it wasn't given. This is what makes the pipeline replayable and testable in isolation.

10. LLM Provider Layer

The platform never calls a model vendor's SDK directly from application code. Every LLM call goes through a single internal interface, and the specific provider behind it is a configuration detail, not an architectural dependency.

This matters for a public-education platform in particular: a hard-coded provider is a single point of both cost and availability risk. If one vendor raises prices, changes rate limits, or has an outage, the pipeline should keep running against a different provider without a code change.

Diagram 9 — Skills call a single LLM provider interface that fans out to any configured model.

Every Skill calls one interface — `ILlmProvider.complete(prompt, response_schema)` — and never imports a vendor SDK directly. Swapping or mixing providers per agent (for example, a cheaper model for storyboard prompts, a stronger one for pedagogy) becomes a configuration change, not a code change. This also gives the project a credible offline story through Ollama, which matters for schools with unreliable connectivity to commercial APIs.

11. Prompt Architecture

Prompts are versioned engineering assets — YAML files in source control, not strings buried in Python.

Diagram 10 — From agent decision to infrastructure execution, every step is typed and validated.

Every prompt call ends in a Pydantic model, not a free-text string. If the model's output doesn't validate, the pipeline fails that node explicitly and reports the error — it does not guess or silently continue with malformed data. Once the shared state is updated, the next agent in the graph either issues another prompt or hands off to the MCP Execution Layer to run a concrete tool (TTS, rendering, encoding) against the infrastructure underneath it.

Skills Manifest

Skill	Responsibility
`CurriculumSkill`	Structures raw chapter text into concept blocks.
`LessonPlanningSkill`	Applies pacing and grade-level constraints.
`TeacherSkill`	Converts scientific terms into localized, student-friendly language.
`StoryboardSkill`	Produces per-scene visual prompts.
`NarrationSkill`	Drives TTS and validates timing output.
`SubtitleSkill`	Builds SRT files from word timestamps.
`RenderingSkill`	Coordinates track placement for MoviePy.
`PublishingSkill`	Finalizes and registers the output asset.

12. MCP Execution Layer

Every external dependency — TTS engine, document parser, video encoder — sits behind the same async tool contract.

Diagram 11 — All tool adapters implement a single IMcpTool interface.

Tool	Wraps
`MarkItDownTool`	Microsoft MarkItDown — PDF to Markdown.
`EdgeTtsTool`	Edge TTS — text to speech.
`MoviePyTool`	MoviePy — multi-track composition.
`FFmpegTool`	FFmpeg — final encode to streaming MP4.
`StorageTool`	aiofiles — non-blocking disk I/O.

`FutureImageGenerationTool` and `FutureTranslationTool` are placeholder interfaces only — they exist so future contributors have a slot to fill, not because they're implemented today.

13. Design Patterns

Every pattern used in this codebase earns its place by solving a real problem the system has today. Patterns that don't clear that bar are deliberately left out, in keeping with the KISS and YAGNI principles this document enforces everywhere else — a pattern with no present-day job to do is just extra structure to maintain.

In use — solving a real problem today

Pattern	Where	Why it earns its place
Adapter	MCP tool wrappers	Bridges third-party binaries (FFmpeg, MarkItDown) into one async contract — without this, every tool integration is bespoke.
Repository	Isar / Dio data layer	Keeps caching and networking details out of business logic — required for offline support, not optional.
Dependency Injection	FastAPI `Depends()` / Riverpod	Needed for testability; the codebase can't hit 80% coverage without it.
Facade	Skills layer	Gives Agents one clean method to call instead of reaching into MCP tools directly.
Observer	Riverpod notifiers / SSE	The entire progress-bar UX depends on this propagation model.

Deliberately not used — not required at current scope

Pattern	Why it's not needed yet
Factory	A simple if/match on tool name is enough at this tool count (5). Revisit only if the tool count grows past ~10.
Strategy	There is currently one grade level and one language. Formalize this once a second locale or grade actually exists — building it now is speculative.

Pattern	Why it's not needed yet
Command	`IMcpTool.execute(**kwargs)` already gives this behavior implicitly; a separate Command layer adds indirection without new capability.
Builder	MoviePy's own timeline API is already a builder. Wrapping it again is redundant.
Template Method	LangGraph's node contract already enforces a consistent execution shape. A base class on top of that is duplicate structure.

This is not a rejection of these patterns — it's a recognition that none of them solve a problem the codebase has today. Adding them now would be exactly the kind of premature structure the platform's own "Simple over clever" principle warns against.

14. Folder Structure

Backend

```
backend/app/
├── main.py
├── core/                    # config, errors, security
├── features/video_generator/
│   ├── router.py            # routes + SSE
│   ├── models.py            # Pydantic state
│   ├── graph.py             # LangGraph construction
│   ├── agents.py            # agent nodes
│   ├── mcp_tools.py         # tool adapters
│   ├── skills.py            # business logic + prompt flow
│   └── skills/               # YAML prompt templates
└── infrastructure/          # storage.py, tracking.py
```

Frontend

```
frontend/lib/
├── main.dart, app.dart
├── core/                              # network, storage, theme, shared widgets
└── features/video_generator/
    ├── data/                          # datasources, models, repositories
    ├── domain/                        # entities, repository contracts, usecases
    └── presentation/                  # providers, screens, widgets
```

15. API Contract

POST /api/v1/videos/generate

Request:

```
{
  "class_level": "6",
```

```
  "subject": "Science",
  "chapter_title": "The World of Plants",
  "file_storage_path": "uploads/chapters/science_ch4.pdf"
}
```

Response (202 Accepted):

```
{
  "task_id": "job_984321_alpha",
  "status": "QUEUED",
  "estimated_time_seconds": 120.0,
  "created_at": "2026-07-11T12:00:00Z"
}
```

GET /api/v1/videos/stream/{task_id}

`text/event-stream` response:

```
data: {"progress": 15.0, "current_node": "ParseDocument", "status": "PROCESSING"}
data: {"progress": 40.0, "current_node": "PlanPedagogy", "status": "PROCESSING"}
data: {"progress": 100.0, "current_node": "END", "status": "COMPLETED", "output_url": "/static/
```

16. Deployment Architecture

Diagram 12 — Request path from the mobile client through the load balancer to observability.

17. Observability

Diagram 13 — Every graph node execution reports spans to the Phoenix trace collector.

```
def configure_telemetry_provider(endpoint_target: str) -> None:
    """Route all traces to the Arize Phoenix collector."""
    provider = TracerProvider()
    provider.add_span_processor(SimpleSpanProcessor(OTLPSpanExporter(endpoint=endpoint_target,
    trace.set_tracer_provider(provider)
```

Every graph node execution emits latency, token usage, and validation-failure spans automatically — this is not optional instrumentation added later.

18. Engineering Standards

Non-negotiable rules. A pull request that breaks any of these is rejected automatically:

- **SOLID.** No low-level library (MoviePy, FFmpeg) is called directly from a route handler — always through an interface.
- **DRY.** Shared logic (file reads, validation) lives in one utility, not copies.

- **KISS.** The pipeline stays linear. No agent routing complexity where a sequential pass works.
- **YAGNI.** No vector database, no semantic cache, no multi-tenant permissions during this MVP.

Python

ruff + black (120 char line length), mypy --strict, Google-style docstrings, pytest + pytest-asyncio at ≥80% coverage. Every I/O path is async.

Flutter

flutter_lints enforced; no raw var/dynamic in business logic. Async state changes go through Riverpod notifiers, not manual triggers. All custom widgets declare `const Widget({super.key})`.

Definition of Done

Compiles clean → tests pass at coverage bar → formatters pass in pre-commit → new code paths emit telemetry → peer review confirms architectural compliance.

19. Sprint Plan (7-Day Hackathon)

Day	Focus
1	API/SSE contracts; Flutter Clean Architecture scaffolding
2	PDF extraction (MarkItDown); Flutter theme and shared atoms
3	LangGraph chain wiring; Riverpod state integration
4	Audio synthesis, MoviePy composition, local storage hydration
5	Native video player UI; route setup
6	Telemetry integration; end-to-end stress testing
7	Performance polish and final submission

20. Roadmap

Phase 2 — Knowledge Retrieval. Add a vector store (Qdrant or PGVector) and RAG to pull context from supplementary reference material across chapters.

Phase 3 — Conversational Tutor. Move from a linear pipeline to a cyclic graph so students can pause and ask follow-up questions, with voice-to-voice interaction.

Both phases are intentionally deferred — building them into the MVP would violate the YAGNI principle this document enforces everywhere else.

21. Architecture Decision Record Summary

ADR	Decision	Status	Rationale
ADR-001	Linear LangGraph over autonomous agent loops	Accepted	Predictable, no infinite-loop cost risk.
ADR-002	File-system video cache, not cloud object storage	Accepted	Avoids multi-region latency overhead during MVP.
ADR-003	Riverpod over BLoC	Accepted	Less boilerplate, compile-safe state.
ADR-004	Flutter for the client	Accepted	60 FPS on low-end Android from one codebase.
ADR-005	FastAPI async backend	Accepted	Native fit with the Python AI/media tooling.
ADR-006	MCP as the tool standard	Accepted	Clean separation of orchestration from tool binaries.
ADR-007	MarkItDown for document extraction	Accepted	Removes custom PDF-to-text parsing work.
ADR-008	MoviePy + FFmpeg for rendering	Accepted	Fast multi-track assembly without a GPU stack.
ADR-009	Async-first standard across the backend	Accepted	Protects the API loop from stalling on long jobs.
ADR-010	No vector DB / RAG in MVP	Accepted	Keeps scope to single-chapter, in-memory state.
ADR-011	LLM Provider abstraction layer	Accepted	Avoids vendor lock-in on any single model provider; enables local/offline inference via Ollama.

22. Known Risks and Mitigations

Risk	Likelihood	Impact	Mitigation
CPU-bound rendering (MoviePy/FFmpeg) blocks worker capacity under concurrent load	Medium	High	Cap concurrent render jobs per worker; queue overflow requests rather than degrading response time.
Single-vendor LLM outage stalls all content creation	Medium	Medium	Mitigated by the LLM Provider Layer (Section 10) — a fallback provider is configured before launch.
No automated retry/backoff on tool failures (TTS, encoder)	Medium	Medium	Add bounded retry with exponential backoff at the MCP tool layer for transient failures only.
80% coverage bar is easy to satisfy with shallow tests	Low	Medium	Require coverage on the agent/skill transformation logic specifically, not just route handlers.
Vernacular/localization quality depends entirely on prompt tuning, with no evaluation	Medium	High	Before scaling past the pilot chapter, add a small human-reviewed sample set to catch

Risk	Likelihood	Impact	Mitigation
harness			localization drift.

None of these risks block the initial build. They are operational hardening items scheduled for the final two days of the sprint and the period immediately following the first release.

23. Closing Note

This document is the implementation baseline for the platform. It reflects the architecture the team has committed to for the hackathon build: a linear, deterministic pipeline that keeps AI cost on the content-creation side and delivers free, offline-friendly playback to every student.

Changes to layer boundaries, the AI orchestration model, or the LLM Provider abstraction should be discussed as a team before they're made. Changes to prompt content, UI copy, or individual Skill implementations can proceed at the engineer's discretion within the standards set out in Section 18.